

# 07

SKT K-뉴딜 아카데미 · AI·  
자동화 기초

## LLM(엘엘엠) · 프롬프트

LLM 을 API(에이피아이) 로 호출해 로그 한 줄의  
설명을 문장으로 받습니다.

프롬프트로 답의 형식을 정하고, JSON(JavaScript  
Object Notation · 제이슨) 으로 받아 파이썬에서  
사용합니다.

# 1교시 — 무료 Gemini(제미나이) API 키를 발급받습니다

1006\_morning30.md 를 열고 스스로 진행합니다. 키가 없으면 2교시 실습을 할 수 없습니다.

| 단계           | 할 일                                          | 되면 보이는 것         |
|--------------|----------------------------------------------|------------------|
| ① 터미널 확인     | VS Code(브이에스 코드) 터미널이 Git Bash(깃 배시) 인지 확인   | 첫 줄에 MINGW64     |
| ② 키 발급       | aistudio.google.com › Get API key (개인 gmail) | 키를 복사함           |
| ③ .env       | security-agent-toolkit 에 저장                  | GEMINI_API_KEY=키 |
| ④ .gitignore | 같은 폴더에 .env 한 줄을 적어 저장                       | 건너뛰지 않음          |
| ⑤ curl 확인    | 키를 싣고 모델 목록을 요청                              | "models" 목록      |
| ⑥ 기록         | docs/2026-10-06.md 저장                        | 키는 적지 않음         |

# API 키는 .env 에 두고 .gitignore 로 막습니다

.gitignore 는 .env 가 깃허브에 올라가지 않게 막습니다.

깃허브에 한 번 올라간 키는 파일을 지워도 기록에 남습니다.

| 파일                  | 적는 한 줄           | 하는 일                                        |
|---------------------|------------------|---------------------------------------------|
| .env (닷 엔브)         | GEMINI_API_KEY=키 | 키를 코드 밖에 보관함. 등호 앞뒤에 빈칸 없음                  |
| .gitignore (깃 이그노어) | .env             | 폴더를 깃허브에 올릴 때 .env 를 빼게 함. 이유는 10/7 에 배움    |
| 주의                  | 웹 화면 업로드         | .gitignore 는 웹 업로드를 막지 못함. .env 를 끌어다 놓지 않음 |

# curl 로 모델 목록을 요청해 키가 동작하는지 확인합니다

터미널에 입력합니다. 줄 끝의 \ 는 명령이 다음 줄에 이어진다는 표시입니다.

```
curl -H "x-goog-api-key: 내_키" \  
      "https://generativelanguage.googleapis.com/v1beta/models"
```

| 보낸 요청                           | 나오는 것                     |
|---------------------------------|---------------------------|
| ① -H(header · 요청 헤더) 로 키를 실어 보냄 | "models" 로 시작하는 긴 목록      |
| ② 키의 마지막 글자를 지움                 | UNAUTHENTICATED — 틀린 키    |
| ③ -H 를 빼고 주소만                   | PERMISSION_DENIED — 키가 없음 |

## 2교시에 사용하는 용어는 네 개입니다

| 용어 (읽는 법)                        | 한 줄 뜻                                              |
|----------------------------------|----------------------------------------------------|
| LLM (엘엘엠 · Large Language Model) | 다음에 올 토큰을 확률로 예측하는 언어 모델                           |
| 토큰 (token)                       | 모델이 문장을 나누는 최소 단위. 사용량과 한도를 토큰으로 센다                |
| 환각 (hallucination · 할루시네이션)      | 사실이 아닌 내용을 사실처럼 생성하는 현상                            |
| API 키 (에이피아이 키)                  | API 를 호출하는(코드로 서비스에 요청을 보내 답을 받는) 사용자를 확인하는 비밀 문자열 |

# 탐지 룰은 조건으로 찾고, LLM 은 문장으로 설명합니다

탐지 룰 — 사람이 작성한 조건으로 찾는다

조건에 맞는 로그만 찾습니다. 예: 로그인  
실패 3회 이상

조건에 없는 로그는 찾지 못합니다

LLM — 로그를 읽고 문장으로 설명한다

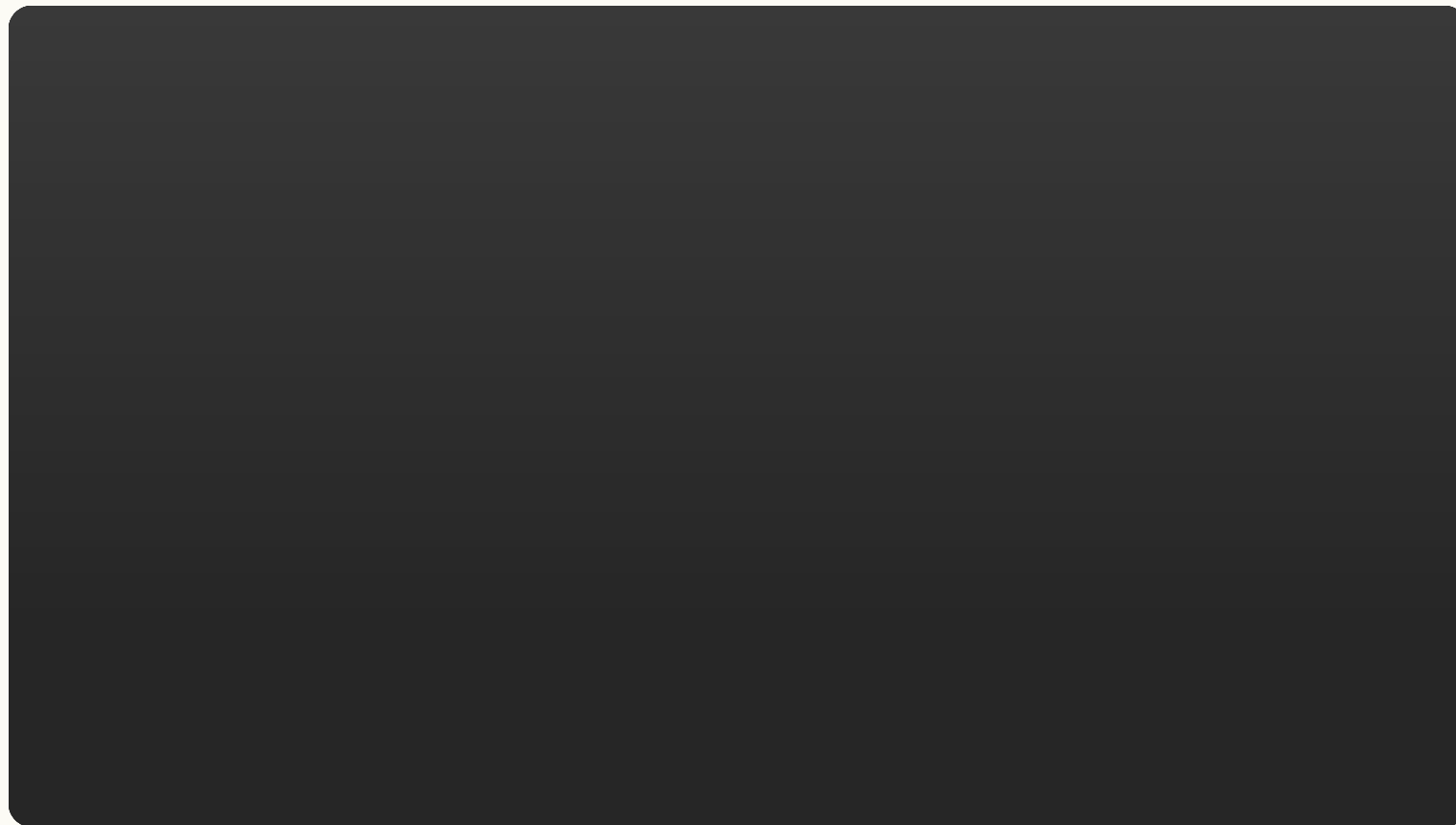
로그 한 줄을 입력하면 무슨 일인지  
문장으로 답합니다

실제 답: 「198.51.100.7 아이피에서  
svc\_batch 계정으로 로그인을  
시도했으나 실패했습니다.」

LLM 을 호출하는 방법은 새로 배우지 않습니다. 앞에서 배운 `requests.post()` 로 **POST**(포스트 · 서버에 데이터를 보내는 요청) 요청을 보냅니다.

# 영상으로 LLM 의 답 만들기를 봅니다 (6분 35초)

LLM 은 다음에 올 단어를 확률로 고르는 일을 되풀이해 문장을 만듭니다.



# LLM 은 다음에 올 토큰을 확률로 예측합니다

① 질문

「한 문장으로 답하세요.  
피싱이란 무엇입니까?」



② 토큰으로 나눔

15토큰



③ 다음 토큰을 하나씩

예측

답 35토큰

LLM 은 사실을 조회하지 않습니다. 학습한 문장을 바탕으로 다음 토큰을 예측합니다.  
그래서 사실이 아닌 답을 만들 수 있습니다. 이 현상이 환각입니다.

토큰 수는 응답의 usage 에서 확인합니다.

질문 15 + 답 35 = 50토큰 — 답 토큰 수는 실행할 때마다 달라집니다.

무료 한도와 요금은 토큰 수로 계산합니다.

# 영상으로 API 가 무엇인지 봅니다 (앞 3분 9초)

API 는 프로그램끼리 데이터를 주고받으려고 미리 정해 둔 방법입니다.



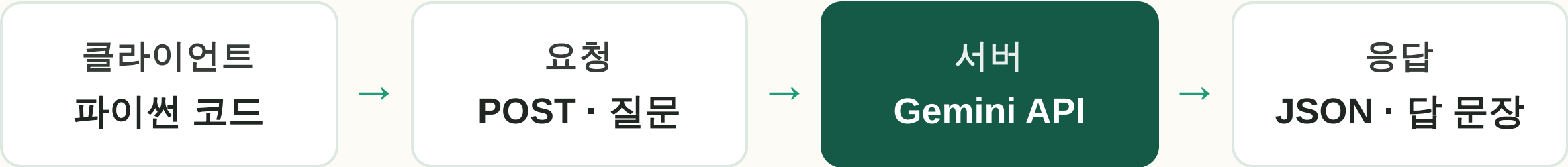
# POST 요청은 서버에 데이터를 보내 처리를 요청합니다

```
requests.post("https://example.com/login",  
              json={"user": "hong", "password": "1234"})
```

| 적은 것     | 뜻                       |
|----------|-------------------------|
| URL(유알엘) | 데이터를 받을 서버 주소           |
| post     | 데이터를 보내는 방식(POST · 포스트) |
| json=    | 서버에 전달할 내용              |

GET 은 「자료를 주세요」, POST 는 「이 데이터를 받아서 처리해 주세요」입니다.

# LLM 호출은 URL(유알엘) · 헤더 · 본문을 지정한 POST 요청입니다



| 지정하는 것 | 넣는 내용             | 오늘 쓰는 값                 |
|--------|-------------------|-------------------------|
| URL    | Gemini API 서버의 주소 | …googleapis.com         |
| 요청 헤더  | API 키             | Authorization: Bearer 키 |
| 요청 본문  | 모델 이름과 질문         | model · messages        |

# .env 파일에서 API 키를 읽습니다

```
api_key = None
with open(ENV_PATH, encoding="utf-8") as f:
    for line in f:
        parts = line.strip().split("=", 1) # = 에서 둘로 나눈다
        if parts[0] == "GEMINI_API_KEY":
            api_key = parts[1] # 뒷부분이 키
```

|                            |                                              |
|----------------------------|----------------------------------------------|
| open(ENV_PATH)             | .env 파일을 엽. ENV_PATH 는 위치 확인 셸이 찾아 둔 경로      |
| line.strip().split("=", 1) | 한 줄을 = 에서 둘로 나눔. 결과는 ["GEMINI_API_KEY", "키"] |
| parts[1]                   | 둘로 나눈 것의 뒷부분. API 키                          |

# 요청 헤더의 **Authorization( 인증 정보)** 에 API 키를 넣습니다

Bearer(베어러 · 가지고 있는 사람)는 「이 키를 가진 사람의 요청입니다」 라는 표시입니다. 키 바로 앞에 붙입니다.

```
headers = {"Authorization": "Bearer " + api_key,  
           "Content-Type": "application/json"}
```

|                 |                                             |
|-----------------|---------------------------------------------|
| "Authorization" | 인증 정보를 넣는 요청 헤더                             |
| "Bearer " (베어러) | 뒤에 <b>공백 한 칸</b> 이 있음. 공백이 없으면 서버가 키를 읽지 못함 |
| api_key         | .env 파일에서 읽은 키. 코드에 직접 쓰면 깃허브에 함께 올라감       |

# 요청 본문에 모델 이름과 질문을 넣습니다

```
body = {  
  "model": "gemini-3.5-flash-lite",  
  "messages": [{"role": "user", "content": "피싱이란 무엇입니까?"}]}
```

|                |                            |
|----------------|----------------------------|
| "model"        | 사용할 모델의 이름                 |
| "messages"     | 질문을 담은 리스트. 안에 딕셔너리 하나를 넣음 |
| "role": "user" | 사용자의 질문이라는 표시              |
| "content"      | 질문 문장                      |

# requests.post() 로 LLM 을 호출합니다

```
import requests      # api_key 는 .env 에서 읽은 값
url = ("https://generativelanguage.googleapis.com"
       "/v1beta/openai/chat/completions")
headers = {"Authorization": "Bearer " + api_key,
           "Content-Type": "application/json"}
body = {"model": "gemini-3.5-flash-lite",
        "messages": [{"role": "user",
                        "content": "한 문장으로 답하세요. 피싱이란 무엇입니까?"]}]}
response = requests.post(url, headers=headers, json=body, timeout=30)
print(response.status_code) # 200 이면 성공
```

200

# 응답을 딕셔너리로 바꾸고 키와 번호로 답 문장을 꺼냅니다

```
data = response.json() # 딕셔너리로 바꾼다  
answer = data["choices"][0]["message"]["content"]
```

|                                     |                                    |
|-------------------------------------|------------------------------------|
| <code>response.json()</code>        | 응답 본문을 딕셔너리로 바꿈                    |
| <code>["choices"][0]</code>         | choices 는 리스트. 숫자 0 으로 첫 번째 값을 꺼냄  |
| <code>["message"]["content"]</code> | 딕셔너리의 키로 꺼냄. content 가 <b>답 문장</b> |

# 실제 응답에서 답 문장을 꺼냅니다

```
data = {  
    "choices": [  
        {"message": {"role": "assistant",  
            "content": "피싱이란 신뢰할 수 있는 기관이나 사람으로 위장하여 "  
                "타인의 개인정보나 금융정보를 속여 빼내는 "  
                "전자 금융 사기 수법입니다."}}  
    ],  
}  
print(data["choices"][0]["message"]["content"])
```

---

피싱이란 신뢰할 수 있는 기관이나 사람으로 위장하여 타인의 개인정보나 금융정보를 속여 빼내는 전자 금융 사기 수법입니다.

## usage 에서 사용한 토큰 수를 확인합니다

| 키                 | 뜻              | 「피싱이란 무엇입니까?」 응답의 값 |
|-------------------|----------------|---------------------|
| prompt_tokens     | 질문의 토큰 수       | 15                  |
| completion_tokens | 답의 토큰 수        | 35                  |
| total_tokens      | 질문과 답을 합친 토큰 수 | 50                  |

`data["usage"]["total_tokens"]` 로 꺼냅니다.

답 문장이 길어지면 `completion_tokens` 가 늘어납니다.

답 토큰 수는 실행할 때마다 달라집니다.

# 되풀이되는 코드를 `post_to_llm()` 함수로 묶습니다

```
body = {"model": "gemini-3.5-flash-lite",  
        "messages": [{"role": "user", "content": "피싱이란 무엇입니까?"}]}  
response = post_to_llm(body) # 한 줄로 요청을 보낸다  
print(response.status_code)
```

200

|             |                                                   |
|-------------|---------------------------------------------------|
| 함수가 대신 하는 일 | 키 읽기 · 요청 헤더 만들기 · 요청 보내기                         |
| 우리가 만드는 것   | 요청 본문 body. 질문이 들어 있음                             |
| 돌려받는 것      | 앞 장과 같은 response. .status_code 와 .json() 을 그대로 사용 |

# 무료 키는 1분에 15번까지 호출할 수 있습니다

한도를 넘으면 상태 코드 429 가 반환됩니다. 키가 고장 난 것이 아닙니다.

| 경우             | 할 일                                        |
|----------------|--------------------------------------------|
| 429 가 반환되면     | 1분 기다린 뒤 같은 셀을 다시 실행한다                     |
| 셀을 한 번 실행하면    | 호출 한 번을 사용한다. 반복문으로 세 번 호출하는 문제는 세 번을 사용한다 |
| 같은 셀을 연달아 실행하면 | 한도를 빨리 넘는다. 연달아 실행하지 않는다                   |

# 상태 코드가 200 일 때만 답 문장을 꺼냅니다

| 상태 코드 | 경우             | 응답의 choices                    |
|-------|----------------|--------------------------------|
| 200   | 호출 성공          | 있다 — 답 문장을 꺼낸다                 |
| 400   | 틀린 API 키       | 없다 — 꺼내면 KeyError(키 에러) 가 발생한다 |
| 429   | 1분에 15번 한도를 넘음 | 없다 — 꺼내면 KeyError 가 발생한다       |

if response.status\_code == 200: 으로 먼저 확인합니다.

200 이 아니면 print("실패:", response.status\_code) 로 상태 코드를 출력합니다.

틀린 키로 실행하면 실패: 400 이 출력됩니다.

# LLM 은 조회하지 않고 답을 생성해서 **사실이 아닌 답(환각)**을 만들기도 합니다

| 질문한 대상          | 받은 답                   |
|-----------------|------------------------|
| LLM · 첫 번째 호출   | LG Uplus               |
| LLM · 두 번째 호출   | SK텔레콤                  |
| 조회 API ipwho.is | <b>SamsungSDS Inc.</b> |

질문은 「IP(Internet Protocol · 아이피) 211.45.12.9 는 어느 통신사의 주소입니까?」 입니다.  
LLM 의 답은 호출할 때마다 달랐고 둘 다 틀렸습니다.  
사실 확인은 조회 API 와 로그로 합니다. LLM 에게는 요약과 설명을 맡깁니다.

## 2교시에 배운 것을 정리합니다

| 하는 일              | 코드                                                                      |
|-------------------|-------------------------------------------------------------------------|
| LLM 에 요청을 보낸다     | <code>requests.post(url, headers=headers, json=body, timeout=30)</code> |
| 요청 헤더에 API 키를 넣는다 | <code>{"Authorization": "Bearer " + api_key}</code>                     |
| <u>답 문장을 꺼낸다</u>  | <code>data["choices"][0]["message"]["content"]</code>                   |
| 호출에 성공했는지 확인한다    | <code>response.status_code == 200</code>                                |

실습 ① · 노트북 2교시 절 · 261006\_am\_llm\_prompt.ipynb

## 문제 2-1 ~ 2-5 · 도전 ★ 2-6

맨 위 위치 확인 셀을 먼저 실행합니다

①

준비 완료 가 출력되어야 합니다. .env 를 찾지 못했습니다 가 나오면 노트북을 agent\_core 폴더로 옮깁니다

②

문제 2-1 · 2-2 — 응답 읽기 · LLM 처음 호출하기

· 2-1 은 결과를 먼저 적습니다

③

문제 2-3 ~ 2-5 — 답 문장 · 조회 API 비교 · 상태 코드

429 가 반환되면 1분 기다린 뒤 다시 실행합니다

# 3교시에 사용하는 용어는 네 개입니다

| 용어 (읽는 법)                              | 뜻                      | 3교시의 예                            |
|----------------------------------------|------------------------|-----------------------------------|
| 프롬프트 (prompt)                          | 모델에게 건네는 <b>지시문 전체</b> | messages 에 답는 글                   |
| <b>system</b> · <b>user</b> (시스템 · 유저) | 역할 지시와 이번 질문을 나눠 답는 자리 | "role": "system" · "role": "user" |
| <b>few-shot</b> (퓨샷)                   | 본보기를 예시로 주는 방식         | 예시: {"severity": "high", ...}     |
| <b>temperature</b> (템퍼러처)              | 답을 얼마나 다양하게 낼지 정하는 값   | "temperature": 0                  |

# 로그만 보내면 파이썬 코드가 값을 꺼낼 수 있습니다

답이 영어로, 항목을 여러 개 나열해 길게 오고 모양이 매번 다릅니다.

프롬프트에 역할 · 지시 · 출력 형식 · 예시를 적으면 답의 모양이 정해집니다.

로그 한 줄만 보냈을 때 받은 답

```
WARN failed login for ops.kim from  
203.0.113.61
```

영어로, 항목을 여러 개 나열한 긴 설명

답 271토큰. 길이와 모양이 매번 다름

뒤에 오는 파이썬 코드가 필요한 것

정해진 값 하나 — high · medium · low

정해진 키 이름 — severity(심각도) ·  
summary(요약)

그래야 키 이름으로 값을 꺼낼 수 있음

severity(심각도)는 경보가  
얼마나 위험한지 나타내는 등급입니다

| 값        | 뜻                 | 예                   |
|----------|-------------------|---------------------|
| low      | 낮음, 기록만 하고 넘어가도 됨 | 로그인 실패 1번           |
| medium   | 보통, 시간 날 때 확인     | 같은 계정 실패 5번         |
| high     | 높음, 바로 확인         | 해외 IP 에서 관리자 계정 로그인 |
| critical | 매우 높음, 즉시 대응      | 랜섬웨어 실행 흔적          |

LLM 이 "severity": "high" 처럼 등급을 붙이면, 코드가 이 값을 보고 경보를 보낼지 정합니다.

# messages 에 system 과 user 딕셔너리를 넣습니다

```
log = ("2026-10-06 03:22:40 WARN failed login "  
      "for ops.kim from 203.0.113.61")  
messages = [  
    {"role": "system",  
    "content": "당신은 보안관제 애널리스트입니다. 한 문장으로만 답합니다."},  
    {"role": "user", "content": log}]  
body = {"model": "gemini-3.5-flash-lite", "messages": messages}  
response = post_to_llm(body)
```

|                  |                                                  |
|------------------|--------------------------------------------------|
| "role": "system" | messages[0] · LLM 이 먼저 읽음. 역할과 답하는 규칙(「한 문장으로만」) |
| "role": "user"   | messages[1] · 그다음 읽고 규칙대로 답함. 이번 질문(로그 한 줄)      |

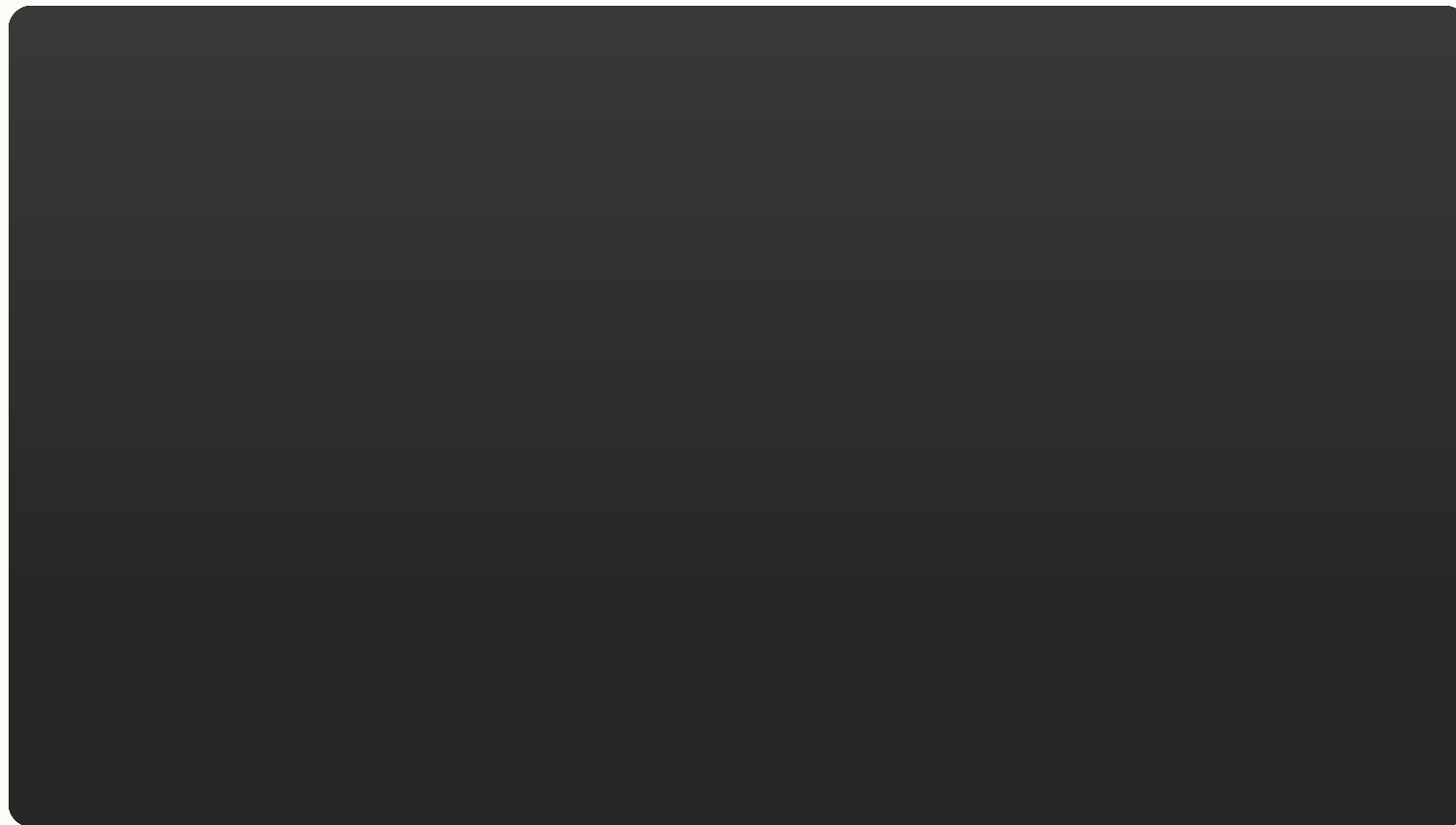
# system 에 적은 역할과 규칙을 **답이 그대로** 따릅니다

같은 ops.kim 로그를 system 없이 한 번, system 을 넣고 한 번 보냈습니다.

|           | system 없음              | system 있음                                                              |
|-----------|------------------------|------------------------------------------------------------------------|
| 보낸 system | 없음                     | 「당신은 보안관제 애널리스트입니다. 한 문장으로만 답합니다.」                                     |
| 받은 답      | 영어로, 항목 여러 개를 나열한 긴 설명 | 「203.0.113.61 IP에서 ops.kim 계정으로의 로그인 실패 (WARN)를 나타내며 ... 모니터링이 필요합니다.」 |
| 답의 길이     | 271토큰                  | <b>한국어 한 문장</b> · 132토큰                                                |

# 영상으로 시스템 프롬프트를 봅니다 (50초)

실제 서비스도 system 자리에 규칙을 길게 미리 적어 둡니다.



# 고를 값을 나열하면 LLM 은 그 안에서만 답합니다

무엇을 할지(지시)와 어떤 모양으로 답할지(출력 형식)를 질문에 적습니다.

```
log = ("2026-10-06 03:22:40 WARN failed login "  
      "for ops.kim from 203.0.113.61")  
instruction = ("다음 로그의 심각도를 high, medium, low "  
              "중 하나로만 답하세요. 로그: "")  
messages = [{"role": "user", "content": instruction + log}]
```

|                          |                             |
|--------------------------|-----------------------------|
| high, medium, low 중 하나로만 | LLM 은 이 세 단어 안에서만 답함        |
| instruction + log        | 지시 뒤에 로그를 붙여 질문 하나로 만들      |
| 받은 답 low                 | 한 단어로 옴. High 처럼 대문자일 때도 있음 |

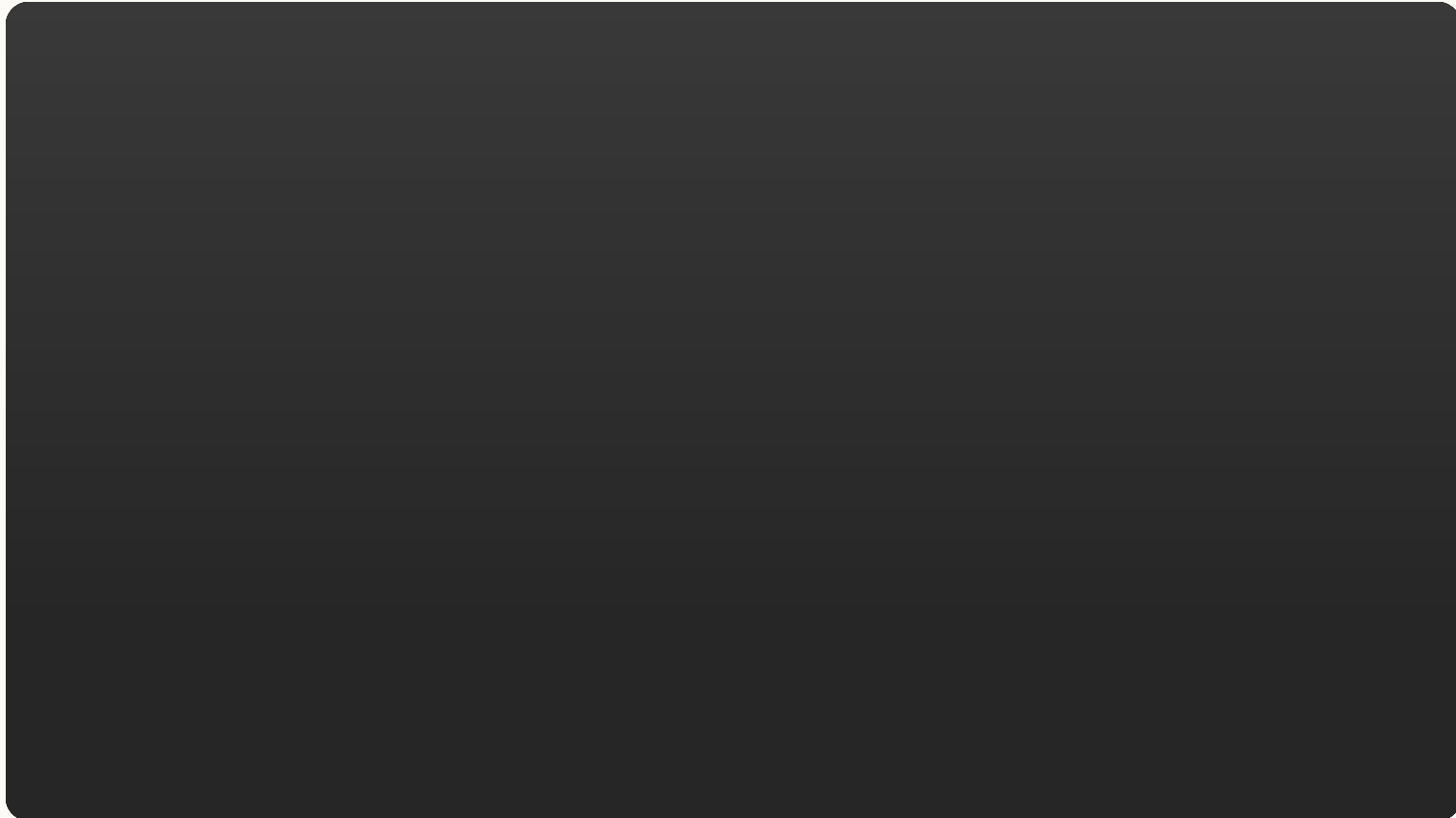
# JSON 으로 물으면 질문에 적은 키 이름으로 답합니다

키 이름을 미리 알고 있으므로 코드가 그 이름으로 값을 꺼낼 수 있습니다.  
앞뒤의 ```json · ``` 은 코드 블록 표시입니다. 4교시에 지웁니다.

```
instruction = ("다음 로그를 분석해 JSON 으로만 답하세요. "  
              "키는 severity 와 summary 들입니다. 로그: "")  
  
print(answer)  
  
```json  
{  
  "severity": "medium",  
  "summary": "IP 주소 203.0.113.61에서 계정 ops.kim으로 로그인 실패 발생"  
}  
```
```

# 영상으로 JSON 이 무엇인지 다시 봅니다 (4분 40초)

JSON 은 프로그램끼리 데이터를 주고받을 때 쓰는 약속된 형식입니다.



# 같은 로그도 묻는 방식에 따라 LLM 의 판단이 달라집니다

두 장 앞과 앞 장은 같은 ops.kim 로그입니다.  
LLM 의 판단은 묻는 방식에 따라 달라집니다.

|          | 한 단어로 물음                              | JSON 으로 물음                                   |
|----------|---------------------------------------|----------------------------------------------|
| 질문에 적은 것 | 「심각도를 high, medium, low 중 하나로만 답하세요.」 | 「JSON 으로만 답하세요. 키는 severity 와 summary 둘입니다.」 |
| 받은 심각도   | low                                   | medium                                       |

# few-shot — 예시를 주면 LLM 이 그 모양을 따라합니다

예시가 짧으면 답도 짧아집니다.

예시의 키 이름 · 순서 · 쓰는 언어도 그대로 따라 합니다.

예시 없음 — 받은 summary

「IP 주소 203.0.113.61에서 사용자 계정 ops.kim으로의 로그인 시도가 실패했습니다.」

예시가 없으면 영어로 올 때가 있음

예시 한 줄을 넣음 — 받은 summary

예시: {"severity": "high", "summary": "root 로그인 실패"}

「ops.kim 계정 로그인 실패」

# 질문 문자열은 지시 · 예시 · 로그 순서로 이어 붙입니다

```
instruction = ("다음 로그를 분석해 JSON 으로만 답하세요. "  
              "키는 severity 와 summary 둘입니다.")  
example = '예시: {"severity": "high", "summary": "root 로그인 실패"}'  
log = ("2026-10-06 03:22:40 WARN failed login "  
       "for ops.kim from 203.0.113.61")  
question = instruction + " " + example + " " + "로그: " + log
```

|                         |                             |
|-------------------------|-----------------------------|
| '예시: {"severity": ...}' | 안에 큰따옴표가 있어서 바깥은 작은따옴표      |
| + " " +                 | 조각 사이에 공백 한 칸을 둬            |
| 지시 → 예시 → 로그            | 무엇을 할지 → 어떤 모양으로 → 무엇을 분석할지 |

# temperature 는 답의 다양함을 조절하는 값입니다

낮추면 답이 안정적이지만, 정답이나 사실을 보장하지는 않습니다.

temperature 낮음 (0)  
답이 비슷하고 안정적



temperature 높음  
표현이 다양하고 예측하기 어려움

|     | temperature 가 조절하는 것 | temperature 가 조절하지 않는 것 |
|-----|----------------------|-------------------------|
| 무엇을 | LLM 이 다음 토큰을 고르는 폭   | 답이 사실인지, 정답인지           |
| 그래서 | 답의 표현과 다양함이 달라짐      | 낮춰도 환각은 사라지지 않음         |

# temperature 는 요청 본문 body 에 넣습니다

```
body = {"model": "gemini-3.5-flash-lite",  
        "temperature": 0, # 보안 자동화에서는 0  
        "messages": messages}
```

|               |                                       |
|---------------|---------------------------------------|
| "temperature" | 답을 얼마나 다양하게 낼지 정하는 값                  |
| 넣는 자리         | messages 안이 아니라 본문 body.model 과 같은 자리 |
| 0             | 답을 가장 덜 다양하게 냄. 보안 자동화에서는 낮게 둬        |

# temperature 0 이어도 문장은 매번 달라집니다

같은 경고 문구 요청을 temperature 0 으로 세 번 보냈습니다.

| 몇 번째 | 받은 답                                              |
|------|---------------------------------------------------|
| 1    | 비밀번호 오류 5회 초과로 계정이 일시 잠겼습니다. 본인 확인 후 잠금을 해제해 주세요. |
| 2    | 로그인 5회 실패로 보안을 위해 계정이 잠겼으니 비밀번호를 재설정해 주세요.        |
| 3    | 비밀번호를 5회 연속 잘못 입력하여 계정이 일시 잠금되었습니다.               |

# 출력 형식을 좁히면 답이 세 단어 안에 모입니다

같은 로그에 high 와 medium 이 섞여 나왔습니다.  
그래서 LLM 의 판단을 그대로 믿지 않고 사람이 최종 확인합니다.

| 로그 (temperature 0 · 세 번씩) | 받은 답 세 개                 |
|---------------------------|--------------------------|
| ops.kim 로그인 실패            | medium · medium · medium |
| root 로그인 실패               | high · medium · high     |

단어는 코드로 세고 비교할 수 있습니다.  
temperature 를 0 으로 해도 환각은 사라지지 않습니다.

# 3교시에 배운 것을 정리합니다

| 프롬프트의 조각     | 어디에 적나            | 무엇이 달라지나                  |
|--------------|-------------------|---------------------------|
| 역할           | messages 의 system | 쓰는 단어와 판단 기준              |
| 지시           | user 의 content    | 무엇을 할지                    |
| <b>출력 형식</b> | user 의 content    | 답의 모양 (한 단어 · JSON)       |
| 예시           | user 의 content    | 답의 길이와 키 이름               |
| temperature  | 본문 body           | 답의 다양함. 0 이어도 답은 달라질 수 있음 |

실습 ② · 노트북 3교시 절 · 261006\_am\_llm\_prompt.ipynb

## 문제 3-1 ~ 3-5 · 도전 ★3-6

- ① 2.4 의 `post_to_llm` 함수 셀을 먼저 실행합니다  
· `NameError` 가 나오면 다시 실행
- ② 문제 3-1 ~ 3-3 — `system` 과 `user` · 역할 넣어 호출 · JSON 으로 받기  
· 3-1 은 결과를 먼저 적습니다
- ③ 문제 3-4 · 3-5 — 예시 · 네 조각과 `temperature 0`  
· ★3-6 은 LLM 을 세 번 호출합니다. 429 면 1분 뒤 다시 실행

# 4교시에 사용하는 용어는 네 개입니다

| 용어 (읽는 법)                                | 뜻                                                 | 4교시의 예                                     |
|------------------------------------------|---------------------------------------------------|--------------------------------------------|
| 파싱 (parsing)                             | 문자열을 <b>프로그램이 쓰는 값</b> 으로 바꾸는 일                   | <code>json.loads(clean)</code>             |
| 코드 블록 표시                                 | 답의 앞뒤에 붙는 <code>```json</code> 과 <code>```</code> | <code>text.replace("`json",<br/>"")</code> |
| <code>call_llm</code> (콜 엘엘엠)            | 질문을 받아 LLM 의 응답 문자열을 반환하는 함수                      | <code>call_llm(question)</code>            |
| <code>parse_llm_json</code> (파스 엘엘엠 제이슨) | 응답 문자열을 딕셔너리로 바꾸는 함수. 못 읽으면 <code>None</code>     | <code>parse_llm_json(text)</code>          |

# 파싱은 정해진 규칙대로 글자를 읽어 프로그램이 쓰는 값으로 바꾸는 일입니다

파일 · API · LLM 에서 받는 것은 모두 글자(문자열)입니다.

| 받은 글자                    | 파싱한 결과               | 쓴 도구              |
|--------------------------|----------------------|-------------------|
| "3"                      | 숫자 3 — 계산 가능         | int()             |
| "03:22:40 WARN"          | ["03:22:40", "WARN"] | split()           |
| "2026-10-06 03:22:40"    | 날짜 · 시각 값 — 시간 차 계산  | strptime()        |
| "from 203.0.113.61"      | "203.0.113.61"       | re.search()       |
| '{"severity": "medium"}' | 딕셔너리 — 키로 꺼냄         | json.loads() · 오늘 |

규칙에 안 맞으면 실패합니다 — int("abc") → ValueError

# 코드 블록 표시가 붙으면 json.loads 가 멈춥니다

3교시에 JSON 으로 답하라고 했더니, 답의 앞뒤에 코드 블록 표시가 붙어서 왔습니다.

```
import json
text = ```json{"severity": "medium"}```
result = json.loads(text)
```

**JSONDecodeError: Expecting value: line 1 column 1 (char 0)**

|                          |                                  |
|--------------------------|----------------------------------|
| json.loads(text)         | JSON 문자열을 딕셔너리로 바꿈 (9/29 오후)     |
| line 1 column 1 (char 0) | 첫 글자에서 멈춤. 첫 글자가 { 가 아니라 ` 이기 때문 |

# 파싱은 JSON 문자열을 딕셔너리로 바꾸는 일입니다

2교시의 `response.json()` 도 파싱입니다.

4교시에는 LLM 이 준 답 문자열을 `json.loads` 로 파싱합니다.

JSON 문자열  
글자일 뿐이라 키로 못  
꺼냄



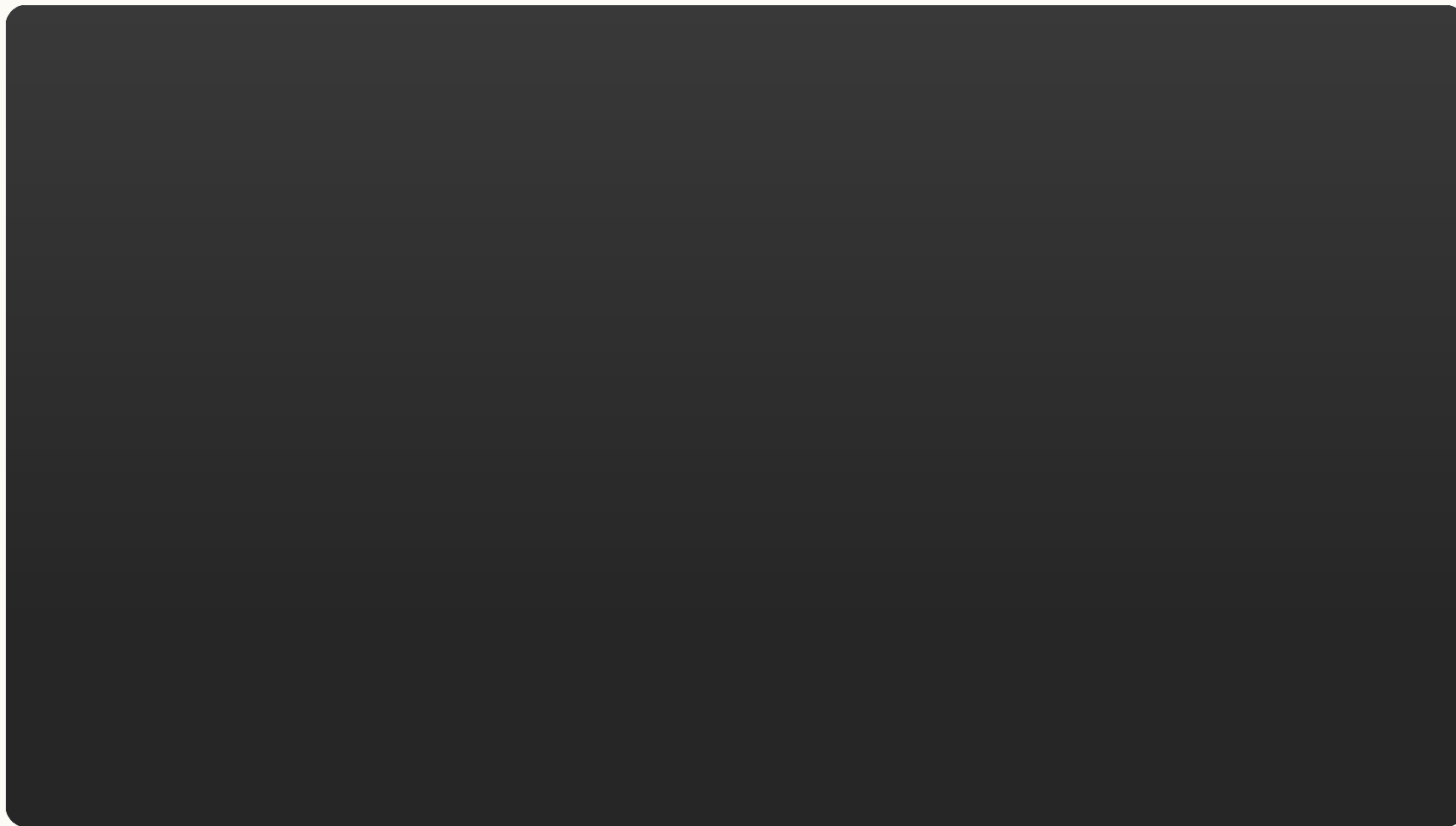
파싱  
문자열을 값으로 바꿈



딕셔너리 · 리스트  
키와 번호로 값을 꺼냄

|         | 2교시 — 응답 본문                                           | 4교시 — LLM 의 답 문장                |
|---------|-------------------------------------------------------|---------------------------------|
| 파싱하는 코드 | <code>response.json()</code>                          | <code>json.loads(clean)</code>  |
| 꺼내는 코드  | <code>data["choices"][0]["message"]["content"]</code> | <code>result["severity"]</code> |

영상으로 파싱을 배워봅시다 (2분 55초)



# json.loads 가 한 건에서 멈추면 남은 아흔아홉 건을 잃습니다

로그 백 건을 차례로 LLM 에 보내 분석합니다.

그중 한 건의 답에 코드 블록 표시가 붙어 json.loads 가 읽지 못했습니다.

|           | 멈추는 프로그램                          | 건너뛰는 프로그램        |
|-----------|-----------------------------------|------------------|
| 읽지 못한 한 건 | <code>JSONDecodeError</code> 로 멈춤 | None 을 담고 다음 건으로 |
| 남은 아흔아홉 건 | 처리하지 못함                           | 계속 처리함           |

4교시에 하는 일 ①

표시를 지우고 읽는다 — `replace · strip`



4교시에 하는 일 ②

그래도 못 읽으면 None 을 담고 계속

# replace 는 글자를 바꾸고 strip 은 앞뒤 공백을 지웁니다

9/23 에 배운 replace 와 9/28 에 배운 strip 을 다시 봅니다.

| 적은 것                                 | 바뀌기 전 → 바뀐 뒤                        | 하는 일                  |
|--------------------------------------|-------------------------------------|-----------------------|
| <code>replace("WARN", "INFO")</code> | "WARN failed..." → "INFO failed..." | WARN 을 찾아 INFO 로 바꿈   |
| <code>replace("WARN ", "")</code>    | "WARN failed..." → "failed..."      | " " 로 바꿈 → <b>지워짐</b> |
| <code>strip()</code>                 | "  로그인 실패  " → "로그인 실패"             | 앞뒤 공백( )만 지움. 가운데는 남음 |

# replace 로 표시를 지우고 strip 으로 공백을 지웁니다

```
import json
text = '`json{"severity": "medium", "summary": "ops.kim 로그인 실패"}`'
clean = text.replace("`json", "")
clean = clean.replace("`", "")
clean = clean.strip()
result = json.loads(clean)
print(result["summary"])
```

ops.kim 로그인 실패

.replace("`json", "")

`json 를 빈 문자열로 바꿔 지움

.strip()

앞뒤에 남은 공백과 줄바꿈을 지움

# ```json 을 먼저 지워야 json 글자가 남지 않습니다

받은 답 ```json{...}``` 에서 두 표시를 하나씩 지웁니다. {...} 는 {"severity": "medium"} 입니다.

|            | 맞는 순서 — 긴 것 먼저                       | 틀린 순서 — 짧은 것 먼저                                          |
|------------|--------------------------------------|----------------------------------------------------------|
| ① 첫 번째로 지움 | replace("```json", "")<br>→ {...}``` | replace("```", "")<br>→ json{...} · ```json 의 ``` 까지 지워짐 |
| ② 두 번째로 지움 | replace("```", "")<br>→ {...}        | replace("```json", "")<br>→ 찾을 것이 없어 json{...} 그대로       |
| json.loads | 딕셔너리로 바뀜                             | JSONDecodeError — 첫 글자가 j                                |

# 읽지 못하면 **try · except** 로 **None** 을 답습니다

JSON 이 아닌 답은 9/28 에 배운 try · except 로 감쌉니다.

```
import json
text = "이 로그는 분석할 수 없습니다."
try:
    result = json.loads(text)
except json.JSONDecodeError:
    result = None
print(result)
```

None

result = None

읽지 못하면 None 을 담고 다음 줄로 넘어감

# parse\_llm\_json 은 딕셔너리나 None 을 반환합니다

응답 문자열을 넣으면 딕셔너리를, 읽지 못하면 None 을 반환합니다.

```
def parse_llm_json(text):  
    clean = text.replace("`json", "")  
    clean = clean.replace("`", "")  
    clean = clean.strip()  
    try:  
        return json.loads(clean)  
    except json.JSONDecodeError:  
        return None  
print(parse_llm_json('`json{"severity": "low"}`'))  
print(parse_llm_json("분석할 수 없습니다."))
```

---

```
{'severity': 'low'}
```

```
None
```

# call\_llm 은 질문을 받아 **응답 문자열**을 반환합니다

```
def call_llm(question):  
    body = {"model": "gemini-3.5-flash-lite", "temperature": 0,  
            "messages": [{"role": "user", "content": question}]}  
    response = post_to_llm(body)  
    if response.status_code != 200:  
        return ""  
    data = response.json()  
    return data["choices"][0]["message"]["content"]
```

return ""

실패하면 빈 문자열을 반환함

→ 다음 parse\_llm\_json 이 멈추지 않고 None 으로 정리

return data[...]["content"]

성공하면 **답 문장**을 반환함

# LLM 호출이 실패해도 `result` 가 `None` 이 되어 멈추지 않습니다

```
text = call_llm(question)          # ① LLM 에게 묻고 답 문자열을 받음
result = parse_llm_json(text)      # ② 답 문자열을 딕셔너리로 바꿈
if result:                         # ③ 바꾸는 데 성공했을 때만 꺼냄
    print(result["severity"])
else:
    print("읽지 못했습니다")
```

| 상황         | ① text                        | ② result            | ③ 화면     |
|------------|-------------------------------|---------------------|----------|
| 잘 됨        | ```json{"severity": "low"}``` | {'severity': 'low'} | low      |
| 한도 초과(429) | "" 빈 문자열                      | None                | 읽지 못했습니다 |
| 엉뚱한 답      | "분석할 수 없습니다."                 | None                | 읽지 못했습니다 |

# %writefile 로 두 함수를 **llm\_client.py** 에 저장합니다

llm\_client.py 는 노트북 없이 혼자 실행돼야 합니다.

그래서 post\_to\_llm 대신 키를 읽는 함수와 requests.post 가 파일 안에 들어 있습니다.

|                           |                                                             |
|---------------------------|-------------------------------------------------------------|
| %writefile llm_client.py  | 셀의 내용을 <b>파일로 저장</b> 함 (9/23 오후). 출력은 Writing llm_client.py |
| URL · MODEL               | 요청을 보낼 주소와 모델 이름을 담은 변수                                     |
| find_env()                | .env 를 지금 폴더부터 다섯 칸 위까지 찾음                                  |
| read_api_key()            | .env 에서 GEMINI_API_KEY 의 값을 읽음                              |
| call_llm · parse_llm_json | 앞에서 만든 두 함수. 빈 줄 두 곳을 채운 뒤 실행함                              |

# import 로 불러 쓰고 if result: 로 확인합니다

같은 폴더의 llm\_client.py 에서 두 함수를 꺼내 쓰고, 꺼내기 전에 None 인지 확인합니다.

```
from llm_client import call_llm, parse_llm_json
instruction = "JSON 으로만 답하세요. 키는 severity 하나입니다. 로그: "
log = ("2026-10-06 02:16:03 INFO accepted login "
      "for root from 198.51.100.23")
text = call_llm(instruction + log)
result = parse_llm_json(text)
if result:
    print(result["severity"])
```

from llm\_client import ...

llm\_client.py 파일에 있는 두 함수를 현재 코드로 가져옴

if result:

result 가 None 이면 꺼내지 않음

# 4교시에 배운 것을 정리합니다

| 하려는 일              | 쓰는 코드                                                              |
|--------------------|--------------------------------------------------------------------|
| 코드 블록 표시를 지운다      | <code>.replace("`json", "").replace("`", "").strip()</code>        |
| 문자열을 딕셔너리로 바꾼다     | <code>json.loads(clean)</code>                                     |
| 읽지 못해도 멈추지 않는다     | <code>try: ... except json.JSONDecodeError: ... return None</code> |
| 질문을 보내 응답 문자열을 받는다 | <code>call_llm(question)</code>                                    |
| 응답 문자열을 딕셔너리로 바꾼다  | <code>parse_llm_json(text)</code>                                  |
| 꺼내기 전에 확인한다        | <code>if result:</code>                                            |

실습 ③ · 노트북 4교시 절 · 261006\_am\_llm\_prompt.ipynb

## 문제 4-1 ~ 4-5 · 도전 ★4-6

- ① 문제 4-1 ~ 4-3 — 표시 지우기 · `parse_llm_json`  
· LLM 을 호출하지 않습니다
- ② 문제 4-4 — `call_llm` 을 담은 `llm_client.py` 저장  
· 4-4 는 2.4 의 `post_to_llm` 셀이 필요합니다
- ③ 문제 4-5 — 두 함수로 로그 한 줄 분석  
· ★4-6 은 LLM 을 세 번 호출합니다. 429 면 1분 뒤 다시 실행